Journal of Intelligent Manufacturing (2023) 34:243–259
https://doi.org/10.1007/s10845-022-02019-7

GPyro: uncertainty-aware temperature predictions for additive manufacturing

Iason Sideris[1,2] · Francesco Crivelli[2] · Markus Bambach[1]

Received: 6 May 2022 / Accepted: 1 September 2022 / Published online: 22 September 2022
© The Author(s) 2022

Abstract

In additive manufacturing, process-induced temperature profiles are directly linked to part properties, and their prediction is crucial for achieving high-quality products. Temperature predictions require an accurate process model, which is usually either a physics-based or a data-driven simulator. Although many high-performance models have been developed, they all suffer from disadvantages such as long execution times, the need for large datasets, and error accumulation in long prediction horizons. These caveats undermine the utility of such modeling approaches and pose problems in their integration within iterative optimization and closed-loop control schemes. In this work, we introduce GPyro, a generative model family specifically designed to address these issues and enable fast probabilistic temperature predictions. GPyro combines physics-informed and parametric regressors with a set of smooth attention mechanisms and learns the evolution of the dynamics inherent to a system by employing Gaussian processes. The model predictions are equipped with confidence intervals quantifying the uncertainty at each timestep. We applied GPyro to Wire-arc additive manufacturing and learned an accurate model from a single experiment on a real welding cell, almost in real-time. Our model can be easily integrated within existing loop-shaping and optimization frameworks.

Keywords Additive manufacturing · Machine learning · Thermal model · Data-driven modelling · WAAM · Uncertainty quantification

Abbreviations

WAAM	Wire-arc additive manufacturing
RBF	Radial basis function
GRU	Gated recurrent unit
ODE	Ordinary differential equation
FEM	Finite element method
PINN	Physics-informed neural network
CNC	Computer Numerical Control
DED	Direct energy deposition
GP	Gaussian process
DTW	Dynamic time warping
MPC	Model predictive control
LSTM	Long short-term memory

✉ Iason Sideris
isideris@ethz.ch

Francesco Crivelli
francesco.crivelli@csem.ch

Markus Bambach
mbambach@ethz.ch

1 Advanced Manufacturing Lab Zurich, ETH Zurich, Rämistrasse 101, Zurich 8092, Switzerland

2 Group for Robotics & Machine Learning, CSEM SA, Gründlistrasse 1, Alpnach 6055, Switzerland

Nomenclature

ρ	Density
λ	Thermal conductivity
h_c, r	Heat transfer coefficient
Q	Heat sinks and sources
Σ	Covariance matrix
θ	Model parameters
z	Physical state
f	Attention mechanism
m	Physics informed regressor
x, y, h	Cartesian coordinates
t_D	Time delay
p	Grid node
x_t	Torch trajectory
l	Loss function
c_p	Specific heat

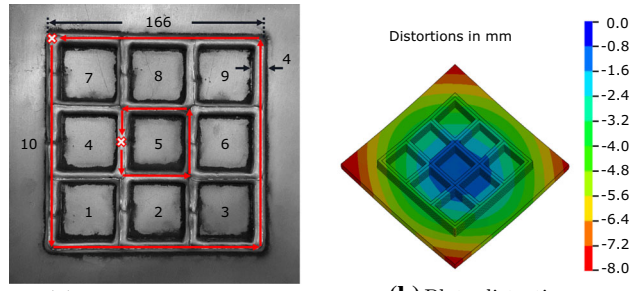
T	Temperature
T_a	Ambient temperature
k	Kernel function
μ	Gaussian mean
s	Model state
F	Model update
g	Surrogate model part
L	Field Laplacians
ξ, η	Grid coordinates
d	Distance
n	Counterpart node
ΔT	Temperature increase
$\hat{e}$	Mean absolute relative error
Index j	Refers to time
Index s	Refers to states
Index f	Refers to component f
Index i	Refers to nodes
Index m	Refers to component m
Index g	Refers to component g

Introduction

Wire-arc additive manufacturing (WAAM) is an additive manufacturing technique used for incrementally building parts by sequentially depositing weld beads on a substrate. This process uses the well established arc-welding technology and is characterized by high deposition rates, reaching several kg/h (Priarone et al., 2020). At deposition time, the feed material in the form of a wire is melted by a high-efficiency heat source usually mounted on a robot or a Computer numerically controlled (CNC) machine. The material transfer is controlled to minimize spatter and achieve the required geometry of the welded tracks.

WAAM is a compelling alternative to conventional manufacturing techniques such as milling due to several reasons. Some of them are the high material utilization, the use of wire raw material, the low-cost equipment needed, the design freedom, and the ability to process high-strength materials. Ideally, this process would allow the production of parts with complicated 3D geometries manufactured by durable alloys with negligible material waste. As a result, customized parts could be fabricated even in small volumes in a sustainable way, offering significant financial benefits to various industries.

However, in practice, distortions and residual stresses (Montevecchi et al., 2016), as well as many defects, such as cracking (Gu et al., 2020), limit the capabilities of WAAM, reducing its utility and undermining the quality of the produced parts. As a result, the process parameters have to be tuned in a trial-and-error manner to achieve an in-specification build. Nonetheless, as trial-and-error requires printing multiple parts to produce a functional one, the material efficiency of the process drops, and thus, its financial advantages are not as attractive as in theory.



(a) Typical WAAM part. **(b)** Plate distortions.

Fig. 1 **a** Typical WAAM part with 9 adjacent pockets, the welding motion pockets 1–9 is the same as for pocket 5; red crosses indicate welding start/stop; pocket 10 indicates contouring motion. In **b**, the measurement of permanent distortion on a cooled 5 mm thick plate is shown, after the material is deposited in the following sequence: 1–2–3–4–5–6–7–8–9–10

These defects originate from many process factors, with the most prominent one being the temperature fields developed during the deposition process (Taylor, 2001). As the heat source deposits molten material on the workpiece, heat is transferred to the part. Then, this heat spreads throughout the workpiece's volume and dissipates into the surroundings. The energy dissipation rate, though, is not equal to the input power from the welding torch; consequently, heat accumulates in the workpiece and the temperature rises over time. The resulting temperature fields are inhomogeneous, and their evolution is strongly coupled with the energy deposition and thus with the printing sequence that the heat source follows.

The heterogeneity in the temperature profiles induces thermal gradients to the produced part. Gradients are strongly related to the formation of defects, and hence, the printing sequence can have an essential role in the final quality of the part (Bambach et al., 2020). For instance, Fig. 1 illustrates how an unsuccessful choice of a material deposition path can lead to a permanent distortion of up to 7 mm within the first 5 printed layers. The severity of this example highlights the impact of the deposition path plan on the final results. However, the magnitude of this impact also indicates that by appropriately designing and optimizing the deposition sequence, one can obtain control over the evolution of the thermal fields.

Designing a path plan that can achieve a specific goal, e.g., the minimization of thermal gradients in a part, requires a model mapping the trajectory of the heat source to the corresponding thermal fields. This model must be fast enough to make simulation-based process optimization a more financially viable solution than the current trial-and-error approach.

There are many ways to approach thermal modeling. The most common approach is using white-box simulations, for

example, Finite element models (FEM). White-box models are helpful for better understanding the effects of the underlying dynamics and, in this way, improving the processes (Xiong et al., 2018). As shown in Nguyen et al. (2021); Graf et al. (2018), which focused on the effects of the thermal fields on the resulting part distortions, these approaches also allow modeling the interactions between coupled physical phenomena. However, finite element methods in additive manufacturing are very computationally expensive and require many hours, even days, to execute. Several approaches as adaptive meshing (Hejripour et al., 2019) or domain decomposition (Neiva et al., 2019) aim to ease this burden, but their computational costs remain high. This problem undermines the efficacy of these approaches, as it results in simulations that last longer than the corresponding experiments and precludes iterative process optimization methods. Furthermore, these models contain a large set of parameters, such as the physical properties of the printed material. These properties have an essential effect on the simulation predictions but their values are usually challenging to determine. This increases the uncertainty of the simulation outputs and can lead to a significant modeling effort.

An alternative to white-box models is offered by their data-driven counterparts, such as Machine Learning models. Machine Learning has offered novel solutions to modeling processes with temporal dependencies. It has been successfully applied in various applications similar to the thermal modeling of additive manufacturing. These methods guarantee comparable accuracy but are significantly faster in comparison to white-box techniques. Moreover, they require fewer assumptions, thus strengthening the robustness of the predictions and casting process automation feasible.

In particular, Mozaffar et al. (2018) used recurrent models, and more precisely Gated recurrent unit (GRU) networks, for studying the temperature evolution of specific part locations in additive manufacturing applications. These architectures have been extensively used for modeling time-dependent data in many applications such as predictive maintenance (de Bruin et al., 2017), natural language processing (Ranzato et al., 2015), etc. They form a broad and powerful family of models with high predictive power and generalization capabilities. Be that as it may, Pascanu et al. (2012) show how recurrent models are notoriously hard to train, with problems like vanishing gradients and lack of memory recitation weakening their performance. It is also usual for such models to require a vast number of training examples for generalizing and lengthy optimization times. This is the case in Mozaffar et al. (2018) on the prediction of temperature profiles in Directed energy deposition (DED) processes as well. This necessity comes in stark contrast to the needs of applications similar to WAAM, where obtaining a model should require as little data as possible, even happening online.

To alleviate the high demand for data, these models are trained and validated on synthetic data produced by simulations. Many high-performance Machine Learning models have been proposed in the literature following this artificial data generation approach, extending from multilayer perceptrons (Le et al., 2022; Roy & Wodo, 2020), graph neural networks (Mozaffar et al., 2021), and extra trees (Ness et al., 2022) to new architectures specialized on the task of thermal predictions (Zhou et al., 2021). However, these solutions usually suffer from a simulation-to-reality gap. The simulated data carry biases and errors introduced by the dynamics incorporated in the simulation, and there is no proof that these models would also retain their performance under real conditions. Moreover, synthesizing these data sets and training on a large amount of data requires many days, even weeks, of effort. In Ren et al. (2020); Pham et al. (2021), the fidelity of the simulations producing the dataset was increased by validating the simulation predictions with measurements coming from real experiments; nonetheless, this is a complicated task that requires extra time and work.

Another issue with the approaches mentioned above is that the majority of them are validated on short time windows of relatively simple and hand-picked geometries with only a few researchers, such as Stathatos and Vosniakos (2019), trying to extend their results for longer prediction horizons. This is usually the case due to the error accumulation that is observed when the models are asked to extrapolate over long periods of time. Long extrapolation times require integrating over many model predictions, leading to even the smallest errors having a substantial cumulative effect.

Furthermore, existing models lack a statistical perspective in their results. Having uncertainty-aware models can be very useful, especially in optimization and control, as it allows to express a notion of confidence about different scenarios and guaranty robustness (Bertsimas et al., 2010). In addition, most of the proposed deep architectures are not well suited for control purposes. While their representation power is very high, they usually result in non-linear and non-smooth functions, making optimization difficult. In fact, Schubnel et al. (2020) emphasizes that it can be more efficient to try to optimize processes described as linear systems based on non-linear regressors rather than using a more precise recurrent network.

In this work, we introduce GPyro (from the Greek word for fire, $\pi\upsilon\rho$), a model that explicitly targets these gaps in the current literature. GPyro is designed to minimize the need for training data and thus allow the optimization with real experiments without compromising the ability to extrapolate over long integration horizons. Moreover, GPyro keeps the structure of the model in a form that is easy to control, it is computationally cheap and it is uncertainty-aware, meaning that we can predict the bounds within which we believe the thermal fields are contained with high proba-

bility. To the best of the authors' knowledge, this work is the first uncertainty-aware model for thermal predictions in additive manufacturing processes that is trained on a single experiment. One can find similar intuition to our approach in Ness et al. (2022), where the authors designed features that would allow the models to generalize without an excessive amount of data and used models suitable for control. However, the authors of this study addressed the problem differently from us. GPyro combines physics-informed and parametric regressors with a set of smooth attention mechanisms in a linear time-variant system and employs Gaussian Processes to predict the evolution of the model parameters through the various stages of the process. In this way, the model is structured in an auto-regressive form, facilitating the formulation of linear control strategies, while learned features capture the non-linearity of the system dynamics.

This work is organized as follows. First, an introduction to the heat equation and to Gaussian Processes can be found in the "Heat equation" and "Gaussian processes" sections, the experimental setup used is found in "Experimental setup" section, then a description of the proposed model with its application in WAAM is illustrated in "Model description" section and, finally, the results, discussion and conclusions in "Results", "Discussion" and "Conclusions" sections. The project's source code can be found at https://github.com/iss1995/gpyro.git.

Preliminaries

Heat equation

Modelling the thermal evolution of workpieces in additive manufacturing processes requires that the heat equation is solved

$$\rho c_p \dot{T} = \nabla \cdot (\lambda \nabla T) + \dot{Q}_{hs} \tag{1}$$

where $T = T(x, y, h, t)$ is the temperature field with x, y, h being the coordinates of the solution domain and t being time, $\lambda(T), \rho(T), c_p(T)$ are the thermal conductivity, the density and the specific heat of the material, respectively, and $\dot{Q}_{hs}$ is a term modeling heat sinks and sources. $\dot{Q}_{hs}$ is usually composed by 2 components for additive manufacturing applications

$$\dot{Q}_{hs} = \dot{Q}_{loss} + \dot{Q}_{source} \tag{2}$$

where $\dot{Q}_{loss}$ are the thermal losses and $\dot{Q}_{source}$ is the thermal input coming from the heat-source. Following (Gh Ghanbari et al., 2020) the loss part $\dot{Q}_{loss}$ includes convection and radiation losses in a single term as

$$\begin{aligned} \dot{Q}_{loss} &= -Ah_c(T - T_a) - A\sigma\epsilon(T^4 - T_a^4) \\ &= -\big(\hat{h}_c + \hat{h}_r(T)\big)(T - T_a) \\ \hat{h}_r &= A\sigma\epsilon\frac{(T^4 - T_a^4)}{T - T_a} = A\sigma\epsilon(T + T_a)(T^2 + T_a^2) \\ \hat{h}_c &= Ah_c \end{aligned} \tag{3}$$

where $\hat{h}_c$ and $\hat{h}_r(T)$ are the effective coefficients of heat transfer, h_c is the convection coefficient, ϵ is the emissivity, σ is the Stefan-Boltzmann constant, A is the free surface of the workpiece and T_a is the ambient temperature. For the the term $\dot{Q}_{source}$ usually volumetric models are employed. One example of such models is the Goldak model

$$\dot{Q}_{source} = \begin{cases} \frac{Pf_f 6\sqrt{3}}{abc\pi\sqrt{\pi}} \exp\big(-3(\frac{\xi'^2}{\alpha^2} + \frac{\eta'^2}{c_f^2} + \frac{\chi'^2}{b^2})\big), & \eta' > 0 \\ \frac{Pf_r 6\sqrt{3}}{abc\pi\sqrt{\pi}} \exp\big(-3(\frac{\xi'^2}{\alpha^2} + \frac{\eta'^2}{c_r^2} + \frac{\chi'^2}{b^2})\big), & \eta' < 0 \end{cases} \tag{4}$$

where P is the heat-source power, a, b, c_f, c_r, f_f, f_r are parameters defining the shape of the volume and χ', ξ', η' is the local coordinate system for the moving heat-source. For more details on this heat-source model the interested reader may refer to Goldak et al. (1984). Eq. 1 excludes the heat flow due to fluid dynamics within the melt-pool and hence, constitutes an incomplete view of the process. Nonetheless, for the scope of this work, this level of approximation suffices. It is important to note that the model parameters are not constant, which means that the process dynamics evolve over time.

For solving the equation, the domain is usually discretized in space, as illustrated in Fig. 3, into several elements defined by a number of nodes on which the thermal field is explicitly calculated. After applying constitutive equations and boundary conditions to all free surfaces, Eq. 1 is transformed to a set of Ordinary differential equations (ODE) following the same procedure as Stockman et al. (2019)

$$\dot{\mathbf{T}}_{\mathrm{j}} = \mathbf{A_T}\mathbf{T_j} + \mathbf{B_T}\mathbf{Q_j} \tag{5}$$

where j indicates the timestep index, $\mathbf{A_T}$, $\mathbf{B_T}$ are the system matrices, and $\mathbf{T_j}$, $\mathbf{Q_j}$ are vectors containing the temperatures and the heat flux at each node during timestep j. A set of ansatz functions is used to interpolate the nodal values and infer the temperatures at locations that do not correspond to the nodes. Eq. 5 shows that the thermal evolution on a grid of points can be approximated by an ODE system, whose parameters depend on the material, the boundary conditions, and the temperature field itself. In this work, we leverage this observation, and we develop a model that learns in a data-driven way an update rule for the $\dot{\mathbf{T}}_j$.

Gaussian processes

Gaussian processes (GP) are widely used in the Machine Learning field to perform non-parametric regression. The goal of non-parametric regression is to find a mapping, $f(\boldsymbol{\theta}) : \mathbb{R}^{dim(\boldsymbol{\theta})} \rightarrow \mathbb{R}$, where $\boldsymbol{\theta}$ is a vector and $f(\boldsymbol{\theta})$ is a scalar. Non-parametric regression in the Gaussian framework considers each variable $f(\boldsymbol{\theta})$ as a random variable, with every fixed set of $f(\boldsymbol{\theta})$ having a joint Gaussian distribution related to the values of $\boldsymbol{\theta}$.

In the literature, there is a wide variety of applications of GP regression in modelling, ranging from drug discovery (Sahli Costabal et al., 2019), materials and molecules modelling (Deringer et al., 2021) to timeseries analysis (Roberts et al., 2013), robotics, control, system identification (Deisenroth et al., 2015; Hewing et al., 2020; Jain et al., 2018) and safe Reinforcement Learning (Berkenkamp et al., 2016, 2017). In the context of WAAM, GPs have been used for weld bead shape optimization (Lee, 2020), but never for tackling the thermal evolution of produced parts. The main attributes that make these models so widespread are the closed-form equations for calculating uncertainty intervals, the easy assimilation of new data, and the ability to include prior knowledge in the models.

To use GP regression, two priors have to be defined for the statistical model; one for the mean and another one for the covariance. The former, $\boldsymbol{\mu}$, is used to reflect a known trend of the process, whereas the latter is the covariance kernel function, k, defining the smoothness and the rate of change of $f(\boldsymbol{\theta})$. With no loss of generality, $\boldsymbol{\mu}$ can be considered zero, meaning that there is no assumption being made regarding the overall trend of f, and the choice of k strongly depends on the modeling problem. Typical kernel choices are the Radial basis function (RBF), the Laplacian, and the Matern kernels. A broader collection of covariance functions and their properties can be found in Rasmussen and Williams (2005).

Each kernel function comes with a set of hyperparameters that tune its behavior. As an example, an RBF kernel function between two vectors, $\boldsymbol{\theta}$ and $\boldsymbol{\theta'}$, is of the form

$$k(\boldsymbol{\theta}, \boldsymbol{\theta'}) = \sigma_n^2 \exp\left(-\frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta'})^T \mathbf{M} (\boldsymbol{\theta} - \boldsymbol{\theta'})\right) + \delta_{\theta,\theta'}\sigma_\omega^2$$

where σ_n is the spread, $\mathbf{M}$ contains the lengthscales of the kernel, $\delta_{\theta,\theta'}$ is the Kronecker δ and σ_ω accounts for measurement noise. In the presence of data $\mathbb{D} = \{\boldsymbol{\theta}, \hat{f}(\boldsymbol{\theta})\}$, the values $\mathbf{a}$ of the hyperparameters can be optimized by applying maximum likelihood type II to maximize the marginal log-likelihood of the data

$$\log p(f(\boldsymbol{\theta})\|\boldsymbol{\theta}, \mathbf{a}) = -\frac{1}{2} f(\boldsymbol{\theta})^T \left(\boldsymbol{\Sigma}(\boldsymbol{\theta}, \boldsymbol{\theta'}) + \sigma_\omega^2 \mathbf{I}\right)^{-1} f(\boldsymbol{\theta}) - \frac{1}{2}\log \|\boldsymbol{\Sigma}(\boldsymbol{\theta}, \boldsymbol{\theta}) + \sigma_\omega^2 \mathbf{I}\| - \frac{1}{2} N \log 2\pi \tag{6}$$

where $\boldsymbol{\Sigma}$ is the covariance matrix and N is the number of datapoints. The optimization of the marginal log-likelihood is performed with gradient methods by using the partial derivatives with respect to $\mathbf{a}$. Eq. 6 is composed of 3 parts. The first part, $-\frac{1}{2} f(\boldsymbol{\theta})^T \left(\boldsymbol{\Sigma}(\boldsymbol{\theta}, \boldsymbol{\theta'}) + \sigma_\omega^2 \mathbf{I}\right)^{-1} f(\boldsymbol{\theta})$, quantifies the wellness of fit to $\mathbb{D}$. The second part, $\frac{1}{2}\log \|\boldsymbol{\Sigma}(\boldsymbol{\theta}, \boldsymbol{\theta}) + \sigma_\omega^2 \mathbf{I}\|$, puts a penalty in complex models and acts as a regularizer, while the last term, $\frac{1}{2} N \log 2\pi$, is a normalization constant.

Having the priors defined, a GP can be used to predict a value $f(\boldsymbol{\theta'})$ based on a set of noisy observations $\mathbb{D}_N = \{\boldsymbol{\theta_i}, \hat{f}(\boldsymbol{\theta_i})\}_{i=1}^N$, with $\hat{f}(\boldsymbol{\theta_i}) = f(\boldsymbol{\theta_i}) + \omega$, and $\omega \sim \mathcal{N}(0, \sigma_\omega^2)$. The mean and the variance of $f(\boldsymbol{\theta'})$ are given by the equations of GP posterior inference

$$\mu_N(\boldsymbol{\theta'}) = \boldsymbol{\mu}(\boldsymbol{\theta'}) + \mathbf{k_N}(\boldsymbol{\theta'}) \left(\boldsymbol{\Sigma}_N + \mathbf{I_N}\sigma_\omega^2\right)^{-1} \hat{\boldsymbol{f}}_N \tag{7}$$

$$\sigma_N(\boldsymbol{\theta'}) = k(\boldsymbol{\theta'}, \boldsymbol{\theta'}) - \mathbf{k_N}(\boldsymbol{\theta'}) \left(\boldsymbol{\Sigma}_N + \mathbf{I_N}\sigma_\omega^2\right)^{-1} \mathbf{k_N^T}(\boldsymbol{\theta'}) \tag{8}$$

where $\boldsymbol{\mu}$ is the prior mean, $\hat{\boldsymbol{f}}_N$ is a vector with all the previous observations of $\hat{f}(\boldsymbol{\theta_i})$, $\mathbf{k_N}(\boldsymbol{\theta'})$ is a vector of N elements with entries $k_N^i(\boldsymbol{\theta'}) = k(\boldsymbol{\theta'}, \boldsymbol{\theta_i})$, $\boldsymbol{\Sigma}_N, \mathbf{I_N} \in \mathbb{R}^{N \times N}$ are the covariance and the identity matrix respectively, whose entries $[\mathbf{6_N}]_\mathbf{j}$ are the vectors $\mathbf{k_N}(\boldsymbol{\theta}'_j)$. After observing a value $f(\boldsymbol{\theta'})$ associated with $\boldsymbol{\theta'}$, the new data can be combined with the rest of the measurements, forming $\mathbb{D}_{N+1} = \{\boldsymbol{\theta_i}, \hat{f}(\boldsymbol{\theta_i})\}_{i=1}^{N+1}$. Then, for inferring the value of $f(\boldsymbol{\theta''})$ for a new vector $\boldsymbol{\theta''}$, the Eq. 7 and. 8 are conditioned on $\mathbb{D}_{N+1}$, assimilating in this way the new data in the model with no extra training. This trait casts the GPs very suitable for online learning. For more details on Gaussian processes we urge the interested reader to refer to Rasmussen and Williams (2005).

Experimental setup

The setup used for performing the training and validation experiments is illustrated in Fig. 2. In each experiment, the deposition of the part shown in Fig. 1a was carried out using different path plans. A mild steel plate of dimensions 250 × 250 × 5 mm was clamped on the positioner. The gas metal arc welding process was carried out with wire of 1 mm diameter, made from mild steel (ER70S-6). While the heat source inputs heat into the workpiece along its welding path $\mathbf{x_t} = \{x_t, y_t, h_t, t\}$, the temperature $T(x, y, h = -0.005, t)$ is recorded on the underside of the support plate. A thermal camera with a resolution of 1024 × 768 pixels at 10Hz is

used to monitor this surface. Monitoring the back of the plate allows to only log the temperature profiles on the observed plane with no disturbances coming from the brightness of the electric arc.

For depositing the parts, a continuous welding motion is chosen for every pocket (1-9) and the contour (10), as indicated in Fig. 1a, always starting and finishing from the same point. This deposition strategy guarantees that the amount of welding starts is low and enables a total of 10! = 3,628,800 path possibilities per layer. Every permutation essentially consists of a different ordering of pocket deposition sequences, e.g. 9,1,2,8,3,7,4,6,5,10. We performed 27 different experiments, each with a different permutation, depositing 6 layers every time. We used 1 of these experiments for training and the rest for validation. The link to the data can be found to the project's repository.

Model description

Model class

To tackle the problem of predicting the evolution of thermal fields in additive manufacturing, which are generated by different paths $\mathbf{x_t}$, a model is needed able to adapt in the time-dependent dynamics of the processes. Furthermore, the model is designed for optimization and control purposes, which constraints the computational costs available for its execution and the complexity of its architecture. To address these issues, a novel generative class of models is introduced, combining physics-informed and simple parametric elements through a smooth attention mechanism. The physics-informed part leverages the prior knowledge relevant to the system, such as Eq. 1, while the parametric parts are used as surrogates to approximate system responses that are not captured by the physics-informed elements. Additionally, the attention mechanism aims to select which parts of the system are activated at each point. To address the variability in the dynamics, the model parameters are smoothly dependent on a system state, i.e., a set of variables that controls the evolution of the dynamics themselves. Firstly, the generic model class is introduced, and then it is adapted to solve thermal problems.

A generic representation of the target model class is

$$F(\mathbf{z}\|\boldsymbol{\theta},\mathbf{s}) = \left(1-\sum_{i}^{M} f_i(\mathbf{z}\|\boldsymbol{\theta},\mathbf{s})\right) m(\mathbf{z}\|\boldsymbol{\theta},\mathbf{s}) + \sum_{i=1}^{M} f_i(\mathbf{z}\|\boldsymbol{\theta},\mathbf{s})\, g_i(\mathbf{z}\|\boldsymbol{\theta},\mathbf{s}),\ \boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}(\mathbf{s}),\ \boldsymbol{\Sigma}(\mathbf{s})) \tag{9}$$

where $m()$ is a (possibly non-linear) physics-informed regressor, constructed based on the balance and constitutional laws describing the physical problem the model addresses, e.g. the energy conservation law, $g_i()$, $i = 1, 2, .., M$ are a collection of parametric models acting as a supplement to $m()$ and $f_i()$, $i = 1, 2, .., M$ are attention mechanisms that activate the correct part of the model at an appropriate time. The model depends on 2 states, $\mathbf{z}$ and $\mathbf{s}$, and a set of parameters $\boldsymbol{\theta}$ which are drawn from a Gaussian distribution, parameterized by a mean function $\boldsymbol{\mu}(\mathbf{s})$ and a covariance matrix $\boldsymbol{\Sigma}(\mathbf{s})$. The different $\boldsymbol{\theta}$ values can be sampled for different values of the state $\mathbf{s}$.

More specifically:

- $\mathbf{z}$ defines the state of the constitutional and balance equations. For instance, in the case of thermal problems, $\mathbf{z}$ would be the temperature distribution in the domain of interest.
- $\mathbf{s}$ defines the state of the system itself. Often, the parameters of the constitutional equations driving a problem change with time. For this reason, the model is treated as a dynamical system, whose state depends on a set of variables. Considering again the example of thermal problems, this change in the dynamics could capture how the material parameters, such as the thermal conductivity $k(T)$, the density $\rho(T)$, and the heat capacity $c_p(T)$ are altered with temperature or how the heat dissipation is affected by the boundary conditions on different parts of the solution domain.
- $\boldsymbol{\theta}$ defines the set of parameters that is used from the models $m()$, $g_i()$ and $f_i()$. $\boldsymbol{\theta}$ also depends on the model state $\mathbf{s}$ as it is a part of the model. In this framework, we define $\boldsymbol{\theta}$ as a GP that models the change in the system's dynamics. GPs are an ideal choice for describing the evolution of the model dynamics as they can effortlessly assimilate new data, such as the parameters $\boldsymbol{\theta}'$ learned for a new state $\mathbf{s}'$ found after optimizing Eq. 9 on a set of new data. Moreover, they provide an easily interpretable statistical perspective in the model class which can help in building robust optimization schemes.

Intuition behind the model: The model is composed of 2 main parts, a physically informed one, $m()$ and some data-driven supplements, $g_i()$, that target what the physical part fails to capture. These two parts are connected by an attention mechanism, $f_i()$, defined as a smooth function, and the sum of all its M components takes values between 0 and 1. This attention mechanism acts like an artificial switch, shifting the model's focus on its right constituents. This switch allows the supplementary models $g_i()$ to only be accurate on the limited values of the state $\mathbf{z}$ that they are needed, making their training easier. Moreover, it decouples the performance of $m()$ and $g_i()$, so that each component does not have to

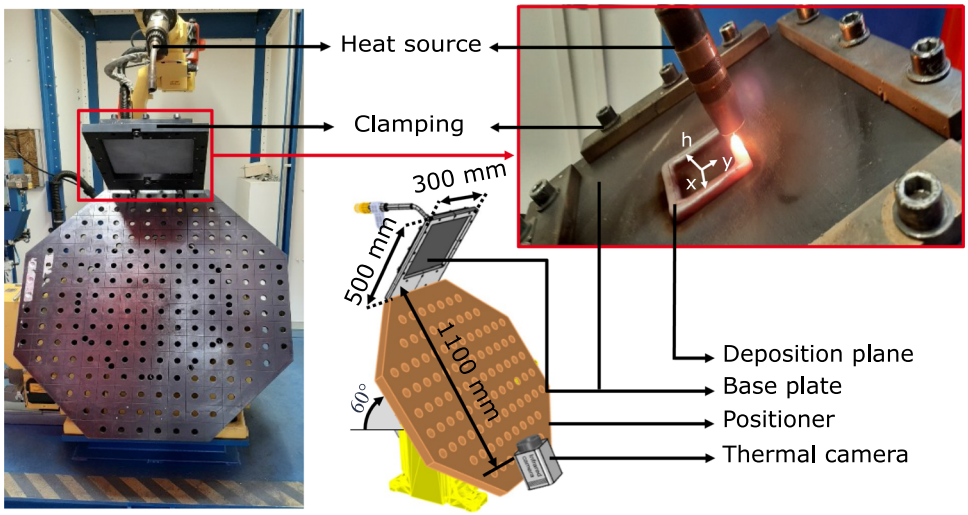


Fig. 2 Welding cell used for data gathering. Used parameters: arc voltage U = 13.1 V; arc current I = 90 A; shielding gas (80%–20%) Ar–CO_2; wire-feed 5.2 m/min; welding speed 6.667 mm/s; feed for non-welding moves 33.333 mm/s

correct the mispredictions of the rest, which would make the training unstable. The trained model would ideally assign a high weight to $m()$ when the physically informed regressor can approximate the system response well and shift its focus to the appropriate $g_i()$ function, when $m()$ fails to follow the correct trend.

Model for WAAM

Equation 9 presents the generic framework of the model class, which can be adapted to the constitutional equations of the problem to be solved. In this work, the model should describe the dynamics of Eq. 1 for parts produced in the welding cell of Fig. 2. To achieve this, the model is adapted to predict the temperature increments anticipated at the different nodes n_i found in Fig. 3a. As the thermal camera in the setup described in "Experimental setup" section can observe the temperatures of the base plate, $T(x, y, h = -0.005, t)$, the learned temperature increments correspond to the observed layer. Confining the problem only to this layer allows using the temperature measurements directly as targets for the model. Extending the model to unobserved layers would introduce inductive biases to the model as no information is given about their state. As a result, the thermal problem we tackle is reduced to a 2D problem.

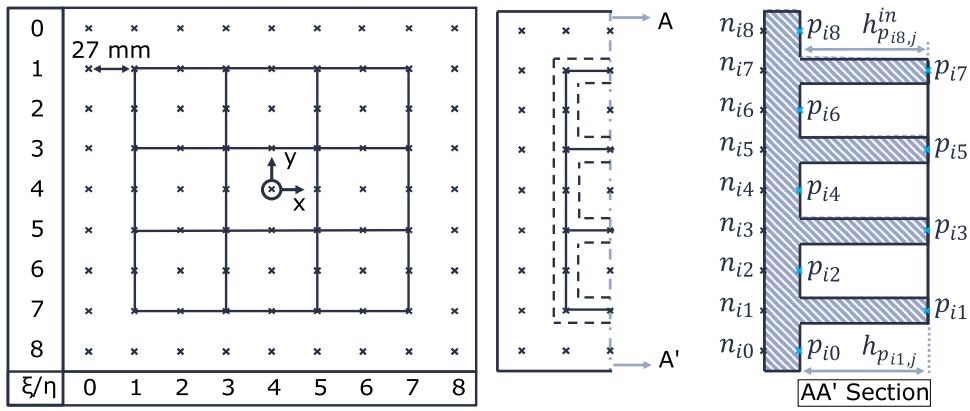
The temperature profiles of the discretized plate, $\mathbf{T_j} = \{T_{i,j},\ i = 0 \ldots n\}$ containing the temperature values of nodes n_i for a timestep t_j, are driven by the dynamics in Eq. 5. They are determined mainly by 3 phenomena, i.e., thermal conduction, heat transfer to the environment, and energy input by the torch. For the 2D case, it is possible to describe the contribution of the former two phenomena based on the plate temperature values, following Eq. 1, since it mainly depends on the temperatures of the observed plane itself. The input heat, though, is not directly observed by the camera. Its effects are only visible through the temperature increase of the base plate after the heat has flown from the deposition layer to the bottom of the base plate. Therefore, heat conduction and dissipation on the plate are incorporated into the non-linear regressor $m()$, and a parametric model $g()$ describes the effect of the heat source on the observed layer. Finally, $f()$ is designed to switch from $m()$ to $g()$ every time the effects of the torch are visible on the plate.

Following this line of thought, the following expressions are derived for each part of the model:

Physics-informed regressor $m()$: This part of the model targets in-plane heat conduction and losses. Firstly, the conduction depends on the Laplacian of the temperature field at every location. Following Eq. 1, for a range of temperatures around T the thermal conductivity can be considered almost constant, $\lambda(T) \approx \lambda$, and the term $\nabla \cdot (\lambda \nabla T) = \lambda \nabla^2 T = \lambda L$ where $L = \nabla^2 T$ are the Laplacians of the thermal field. The vector of Laplacians $\mathbf{L_j} = \{L_{i,j},\ i = 0 \ldots n\}$ is obtained similarly to Stockman et al. (2019) by a second-order centred difference approximation of the sum of the second order spatial derivatives, i.e.

$$\begin{aligned} \mathbf{L_j} &= \partial^2 \mathbf{T_j}/\partial x^2 + \partial^2 \mathbf{T_j}/\partial y^2 \\ L_{i,j} &\approx \sum_{i' \in l} (T_{i',j} - T_{i,j}) / \|\mathbf{x_{i'}} - \mathbf{x_i}\|_2^2 \\ l &= \big\{ i' = \eta' + 9\xi',\ \eta' \in \{\eta - 1, \eta + 1\}, \\ &\quad \xi' \in \{\xi - 1, \xi + 1\} \big\} \end{aligned} \tag{10}$$

where η and ξ are the grid coordinates of each node n_i as illustrated in Fig. 3 and $\mathbf{x_i}$ is the corresponding location given a frame of reference. Thus, $\mathbf{L_j}$ fuses the temperature values of neighboring nodes at each timestep to estimate the heat conduction. Secondly, the energy dissi-



(a) Grid of n_i **(b)** Example of counterparts.

Fig. 3 **a** Selected discretization for the 2D plate. Each node n_i is characterized by $i = \eta + 9\xi$. The distance between neighboring nodes is 27 mm, yielding a sparse grid. Our algorithm allows for sparse grids and linear reduction in computational burden. **b** Example of locations of n_i and p_i, $h_{pi,j}$, $h^{in}_{pi,j}$ shown on a section of the the printed geometry

pation depends on the temperatures of the field, and on the boundary conditions as described in Eq. 3. The temperature fields are considered normalized with respect to the ambient temperature, resulting in $T_a = 0$. As a result, the loss term in Eq. 3 becomes $\dot{Q}_{loss} = -(\hat{h}_c + \hat{h}_r(T))T$. The boundary conditions are not explicitly stated, but learned for each stage of the process. For taking under consideration the boundary conditions at the edges of the plate, a binary vector, $\boldsymbol{\delta_1}$, is introduced, with its components either taking the value 1 or 0. The vector $\boldsymbol{\delta_1}$ is set to 1 for all the locations close to the edge of the plate.
With the introduction of this vector, m takes the form

$$\mathbf{m_j} = m\big(\mathbf{T_j} \| \boldsymbol{\Theta_m}(s_j)\big) = \boldsymbol{\Theta_m}(s_j) \cdot \big[\mathbf{T_j}, \quad \mathbf{L_j}, \quad \delta_1\, \mathbf{T_j}\big] \tag{11}$$

where $\mathbf{s_j} = \{s_{i,j},\ i = 0 \ldots n\}$ is the state of the system at the timestep t_j for the every node n_i, $\mathbf{m_j} = \{m(T_i \| s_{i,j}),\ i = 0 \ldots n\}$ is the vector of calculated $m()$ values corresponding to $\mathbf{s_j}$, and $\boldsymbol{\Theta_m} = \{\theta_{u,i},\ u = 0 \ldots 2,\ i = 0 \ldots n,\}$ are the relevant parameters for each variable. Each element $\delta_{1,i}$ of the vector $\boldsymbol{\delta_1}$ encodes spatial information to the model and acts as a Kronecker δ on the term of Eq. 11, activating and deactivating it, according to the location of each node. Since each regressor term aims to model a physical phenomenon, we can also infer the sign of the corresponding weights. From the law of energy conservation we can deduce that $\boldsymbol{\theta_0}, \boldsymbol{\theta_2}, \leq 0$ and $\boldsymbol{\theta_1}, \geq 0$. These constraints can be used during optimization to accelerate convergence and guarantee that the energy balance is conserved.

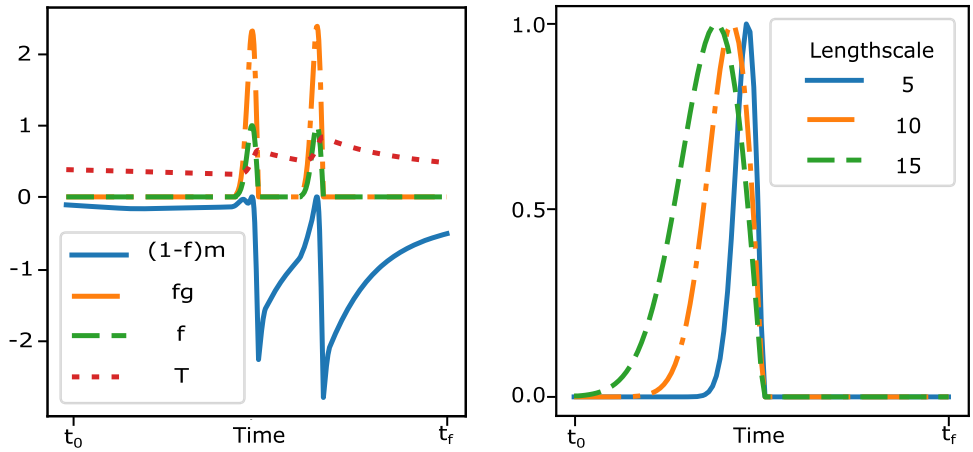
Surrogate model $g()$: This part of the model captures the effect of the moving heat source on the temperature profile observed at the bottom surface of the base plate. This surrogate could be any parametric model, e.g., a neural network. In practice though, since the data and time needed for optimizing the model parameters are to be minimized, a simple linear regression scheme is chosen

$$\begin{aligned}\mathbf{g_j} &= g\big(\mathbf{T_j}, \mathbf{h_{p,j}} \| \boldsymbol{\Theta_g}(s_j)\big) \\ &= \boldsymbol{\Theta_g}(s_j) \cdot \big[\mathbf{1_n} \quad \mathbf{1_n} - \mathbf{h_{p,j}} \quad \mathbf{1_n} - \mathbf{T_j}\big]\end{aligned} \tag{12}$$

where $\mathbf{h_{p,j}} = \{h_{p_i,j},\ i = 0 \ldots n\}$ is the vector of deposition heights, i.e. the distance of the deposited layer from the observed plane as seen in Fig. 3 , $\mathbf{1_n}$ is a vector of ones, and $\boldsymbol{\Theta_g} = \{\theta_{u,i},\ u = 3 \ldots 5,\ i = 0 \ldots n,\}$ are the relevant parameters for each variable. The simplicity of the surrogate model facilitates the optimization by reducing the number of learned parameters as much as possible.

Attention mechanism $f()$: This part of the model selects which component is activated. Considering that $g()$ describes the thermal contribution of the torch, its output should be taken into account when the heat of the deposited material in a neighborhood of a node has traveled through the part. Thus, $f()$ is designed to choose model components according to the distance of the torch from each node and to a heat transport delay.
To calculate both the delay and the distance, the projection, p_i, of each node, n_i, is defined on the deposition plane. The projections p_i are named as the *counterparts* of n_i, and from now on, they are referred to as such. Then, the distances $d_{i,j}$ between p_i and the position of the torch at timestep t_j, $\mathbf{x_{t_j}}$, and the amount of time-steps $t_{D_{i,j}}$ required for the heat to arrive at the observed plane are evaluated. With these 2 values, f can be calculated as

$$\begin{aligned}\mathbf{f_j} &= f\big(\mathbf{d_j}, \mathbf{t_{D_j}} \| \boldsymbol{\theta_f}(s_j)\big) \\ &= \begin{cases} \frac{d}{dt} \exp\left(-\left(\frac{\mathbf{d_j}(\mathbf{1}_n t_j - \mathbf{t_{D_j}})}{\boldsymbol{\theta_f}(s_j)}\right)^2\right) c, & \mathbf{f_j} > 0 \\ 0, & \mathbf{f_j} \leq 0 \end{cases}\end{aligned} \tag{13}$$



(a) Contributions of different model parts

(b) Example of f sequences for different lengthscales.

Fig. 4 **a** Contribution of different components to predicted temperature. Contribution $(1-f)m$ is scaled by a factor of 500 for visibility. Notice how the attention mechanism activates the different components and the coupling between the values of $m()$ and $g()$. **b** Function f for different lengthscales

$$\mathbf{t_{D_j}}(\mathbf{h_{p,j}}) = a_d\,\mathbf{h_{p,j}} + b_d\mathbf{1}_n$$
$$\mathbf{d_j} = \left\|\mathbf{p} - \mathbf{x_{t_j}}\right\|_2 \tag{14}$$

where $\mathbf{d_j} = \{d_{i,j},\ i = 0\ldots n\}$ is the vector of distances between each n_i and the torch position at timestep t_j, $\mathbf{t_{D_j}} = \{t_{D_{i,j}},\ i = 0\ldots n\}$ is the vector of heat transfer delays, c is a normalizer keeping the f value between 0 and 1, $\mathbf{p} = \{(x_i, y_i, h_{p_i}),\ i = 0\ldots n\}$ is the vector of positions of the counterparts of n_i and a_d, b_d are the parameters of the delay model. The parameter $\boldsymbol{\theta_f} = \{\theta_{u,i},\ u = 6,\ i = 0\ldots n,\}$ is a lengthscale that regulates how abrupt the switching is. For simplicity, the nodes n_i where material is not deposited directly obtain a value of $f_{i,j} = 0$.

A visualization of the different model components can be found in Fig. 4a, while the shape of the activation function for different values of θ_f is found in Fig. 4b. Fig. 4a illustrates the contributions and the coupling between the components. After the term g is activated, the physics-informed part m, responsible for heat conduction and dissipation, obtains significantly higher values and the temperature profiles achieved resemble Newtonian cooling. Moreover, it is also interesting to observe the effect of the attention mechanism on the activation and deactivation of the components.

To model the evolution of the system dynamics, states $s_{i,j}$ should be assigned to each node n_i, reflecting the different stages of the process. For facilitating training, the states are expressed as $s_{i,j} = h_{p_{i,j}}$, where $h_{p_{i,j}}$ is the current height of n_i's counterpart. Moreover, different states are assigned to nodes close to boundaries to learn the effect of boundary conditions on the system dynamics. Thus, the nodes n_i that do not get excited and lie in the central plate remain at $s_i = 0$ for every timestep t_j, while the rest evolve. For simplifying the approach and keeping the state of the system dependent on a single variable, the states of the nodes on the boundaries are encoded as $s_{i,j} = -h^{in}_{p_{i,j}}$, where $h^{in}_{p_{i,j}}$ is the current height of the counterpart of the closest node to n_i that does not lie on the boundary, as illustrated in Fig. 3b. Thus

$$s_{i,j} = \begin{cases} h_{p_{i,j}}, & \delta_{1,i} = 0 \\ -h^{in}_{p_{i,j}}, & \delta_{1,i} = 1 \end{cases} \tag{15}$$

Finally, the Gaussian processes that model the dependency of the model parameters $\hat{\boldsymbol{\theta}} = \{\theta_u,\ u = 0\ldots 6\}$ to the system states $\mathbf{s_j}$ are modelled with a bilinear prior mean, and a Matern 3/2 kernel. An important note here is that a bilinear prior mean does not imply a linear posterior mean, as the value of the latter function is defined by Eq. 7 and the calculated weights. Every component of $\hat{\boldsymbol{\theta}}$ is assigned a separate GP regression model. This decouples the weights and allows each model to grow independently.

Training

Optimizing Eq. 9 jointly on the data of a single experiment is not a straightforward task. The interconnection between $f()$ and the rest of the components is not easy to optimize as the optimization can turn unstable. Moreover, even if the scheme is stable, optimizing the product of functions yields a highly complex optimization problem, which is challenging to solve.

In this work, the goal is to achieve learning in almost real-time, and a complicated optimization scheme would not be fit. Thus, to reduce the optimizer's computational burden and remove the solution's dependency on the initialization point, the optimization scheme should be as simple as possible with-

Algorithm 1 Training algorithm: After every deposited layer, the new data are incorporated into the training algorithm, split to their corresponding states, and used to retrain (if needed) each state-model $\boldsymbol{\theta_s}$ separately. Then, the model's point-estimates are used as samples for a Gaussian Process that acts as a generative model for the parameters θ.

Initialize:
$\theta \leftarrow \theta_0,\ b_f,\ b_m,\ N_f,\ N_g,\ N_m$ # $b \rightarrow$ optimization bounds, $N \rightarrow$ Horizon
Inputs:
New data: $x_l^t,\ h_l,\ T_l^i$ # $l \rightarrow$ layer, $i = 1 \ldots n \rightarrow$ nodes, $t \rightarrow$ torch

For each layer:
$T^i \leftarrow T_l^i \cup T^i$ # Collection of temperature profiles.
$H \leftarrow h_l \cup H$ # Collection of heights.
$X^t \leftarrow x_l^t \cup X^t$ # Collection of torch trajectories.
$X_{sync}^t,\ H_{sync},\ T_D \leftarrow \text{Synchronize}(X^t,\ T^i,\ H)$ # $T_D \rightarrow$ delays
$X_S^t,\ H_S,\ T_S^i,\ S \leftarrow \text{SplitToStates}(X_{sync}^t,\ T^i,\ H_{sync})$ # $S \rightarrow$ states
for $\left(x_s^t,\ h_s,\ T_s^i,\ s\right) \in \left(X_S^t,\ H_S,\ T_S^i,\ S\right)$ **do**
  $\theta_{s,f} \leftarrow \text{optimizeF}(x_s^t,\ s,\ T_s^i,\ \theta_{f_0},\ b_f,\ N_f)$
  $\theta_{s,g} \leftarrow \text{optimizeG}(x_s^t,\ s,\ h_s,\ T_s^i,\ \theta_{s,f},\ \theta_{g_0},\ N_g)$
  $\theta_{s,m}, \theta_{s,p} \leftarrow \text{optimizeM}(x_s^t,\ h_s,\ T_s^i,\ \theta_{s,f},\ \theta_{s,g},\ \theta_{m_0},\ \theta_{p_0},\ b_m,\ N_m)$
end for
$a_d, b_d \leftarrow \text{FitDelay}(S,\ H_S,\ T_D)$
$\theta_0 \leftarrow \theta_s$
$\text{GP}\big(\mu(s), \Sigma(s)\big) \leftarrow \theta_s$

out compromising the quality of the solution found. This introduces the need to simplify our optimization approach.

The necessary simplification can be achieved by breaking the joint optimization problem to its constituents. Instead of learning all the parameters $\hat{\boldsymbol{\theta}}(s)$ for every s simultaneously, the optimization focuses on independent sets of $\hat{\boldsymbol{\theta}}_{\boldsymbol{s}}$ for each discrete value or group of s. Then, all the available estimations of $\hat{\boldsymbol{\theta}}_{\boldsymbol{s}}$ are jointly united with a Gaussian Process and predicted at inference time. Therefore, each $\hat{\boldsymbol{\theta}}_{\boldsymbol{s}}$ is considered a sample from the Gaussian random process $\hat{\boldsymbol{\theta}}(s)$, with a mean and a covariance $\boldsymbol{\mu(s)}$ and $\boldsymbol{\Sigma(s)}$. The GP $\hat{\boldsymbol{\theta}}(s)$ can be then sampled to provide the values of parameter $\boldsymbol{\theta}$ for any value of s'.

To learn each $\hat{\boldsymbol{\theta}}_{\boldsymbol{s}}$, 4 sequential steps are executed. Firstly, the thermal data observed at the bottom of the base plate are synchronized with the movements of the heat source. In particular, the synchronization finds and matches the high temperature peaks observed at n_i with the corresponding movement of the heat source that caused them. In practice, the output of the synchronization step is a pair of timesteps including the time that the torch crossed p_i and the time needed for the heat to reach n_i. This allows decoupling the performance of the delay model found in Eq. 14 from the rest of the system.

Secondly, the attention mechanism is trained to select the constituents of the model. Since the role of the attention mechanism $f()$ is to activate the surrogate $g()$ when the heat of the welding torch arrives at a location n_i, $\theta_{s,f}$ is trained to activate $f()$ long enough to capture the whole duration of the heat wave.

After defining $f()$, $g()$ can be optimized on the time segments that $f_j > 0$ to only capturethe increase in the temperature due to the incoming heat wave. Finally, $m()$ is optimized by taking the other two system components into account. For fusing and scaling the predictions of $g()$ and $m()$, an extra parameter, θ_p, is introduced and optimized together with $m()$.

By substituting Eq.11-14 into Eq.9 and by scaling g with $\theta_p(s_j)$, the final form of the thermal model for WAAM is derived

$$\Delta T_{i,j} = \left(1 - f_{i,j}\right) m_{i,j} + f_{i,j}\, \theta_{s,p}\, g_{i,j} \tag{16}$$

for every t_j that a node n_i is in state s. Eq. 16 shows the operations needed to calculate the temperature increment of each node at each timestep. Following this formula, the remaining parameters $\boldsymbol{\theta_{s,m}},\ \theta_{s,p}$ are optimized to describe the temperature increment on each node given the previous temperatures, the torch path, the optimized surrogate and attention mechanisms. The final set of parameters considered by the model is $\boldsymbol{\theta_s} = \hat{\boldsymbol{\theta}}_{\boldsymbol{s}} \cup \theta_{s,p}$, and θ_p is also assigned a GP regression model.

This learning scheme is repeated every time a new batch of data is gathered for every state value s acquiring new measurements. The parameters are updated until the model's predictive performance for s does not change significantly with new data. The loss function used for the optimization is the mean of the square relative error

$$l(\mathbf{e_N}) = \frac{1}{N} \sum_{i}^{N} e_i^2 + l\, \|\boldsymbol{\theta_s}\|_2$$

where N is the prediction horizon used in every optimization step, l is the regularization coefficient, and $\mathbf{e}$ is the relative error of the model. For $g()$ and $f()$ a prediction horizon $N = 100$ timesteps of size $\Delta t = 0.5$ sec was used while $m()$ was optimized on $N = 1$.

Every optimization step can be performed either with auto-differentiation libraries like PyTorch or, due to the low dimensionality of the optimization problem, with out-of-the-self optimizers like l-BFGS-b (Byrd et al., 1995). In this work, we choose the latter approach to prove that even with a mid-range PC, one can train the model states in a few seconds. More specifically, our training scheme needs approximately 2.14 sec for obtaining an estimate of $\boldsymbol{\theta}_{s'}$ for every new state s', running on a single core of an AMD Ryzen 9 5900X, with a core bandwidth of 3,7GHz. For every optimization step 75% of the available data is used for training and 25% is used for validation. A synopsis of the training algorithm is shown in Alg. 1.

Finally, the obtained parameters are included in the GPs after the optimization is finished. For each parameter, a GP regression model is maintained and updated with the new parameters. The hyperparameters of the GPs can be updated as well with the addition of new states by maximizing the Eq. 6.

Model propagation

Once the GP regression models for $\boldsymbol{\theta}(s)$ are obtained, the weight values $\boldsymbol{\Theta}(\boldsymbol{s}_j) = \{\boldsymbol{\theta}(s_{i,j}),\ i = 0 \ldots n\}$ can be sampled for any timestep t_j, determining thus the model in Eq. 16. The states of the system, as defined in Eq. 15, only depend on the path plan. As a result, given an arbitrary predifined torch movement $\mathbf{x}'_\mathbf{t}, \mathbf{s}'_\mathbf{j}$ is defined and the values $\boldsymbol{\Theta}(\boldsymbol{s}'_j)$ can be directly sampled from a Gaussian distribution parametrized by a mean and a covariance matrix defined in Eq. 7 and 8. One reasonable set of samples $\boldsymbol{\Theta}(\boldsymbol{s}'_j)$ one can choose is the mean of the GP, as this represents the most probable weight values learned.

With the estimations of the parameters $\boldsymbol{\Theta}(\boldsymbol{s}'_j)$, Eq. 16 can be used to approximate the temperature increment of each node on a discretized domain. The thermal fields can be reproduced step by step by integrating the thermal increments over time for all nodes. The node-wise operations for each timestep introduced in Eq.16 can be executed in parallel to save computational time. Therefore, the time integration is performed in batches, where each batch contains the updates of each node on the grid. The integration formula is given as

$$\begin{aligned} \mathbf{T_{j+1}} &= \mathbf{T_j} + \boldsymbol{\Delta T}_j \\ \boldsymbol{\Delta T}_j &= \left(\mathbf{1_n} - \mathbf{f_j}\right) \odot \mathbf{m_j} + \mathbf{f_j} \odot \boldsymbol{\theta}_p(\boldsymbol{s}_j) \odot \mathbf{g_j} \end{aligned} \tag{17}$$

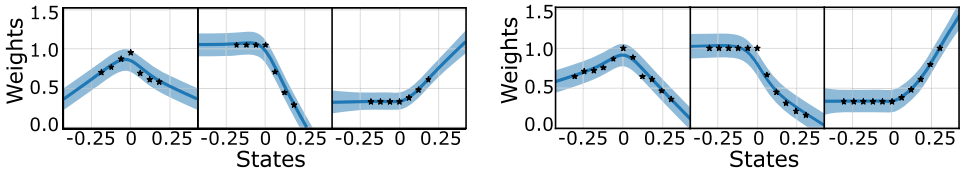
where $\odot$ is the Hadamard product for element-wise multiplication. For updating each node, the model considers the temperatures surrounding the node, the boundary conditions applicable, and any thermal contribution stemming from the torch. The only necessary elements to fully define a temperature extrapolation are an initial temperature field $\mathbf{T_0}$, and a torch path plan $\mathbf{x_t}$ for an horizon N.

Results

Incremental training of Eq. 16 results in having different system representations at every stage of the process. This can be seen in Fig. 5, where the evolution of 3 parameters is illustrated after printing 4 and 6 layers. The model predicts the bounds within which the weights $\boldsymbol{\theta}$ are contained with high probability. The most probable weights are found by calculating the mean of the Gaussian process. An important observation is that the most probable weights are not always identical to the weights stemming from the state-wise optimization, as the GP evaluates the joint probability of all samples for inferring the mean.

With the estimations of the parameters $\boldsymbol{\theta}$, Eq. 16 can be used to approximate the solution of the heat equation. By using the mean of the GP to define the most probable $\boldsymbol{\theta}$ and integrate as described in "Model propagation" section, the thermal fields of the validation set are reproduced. For instance, Fig. 6 illustrates the thermal fields derived after integrating 1650 steps (825 sec) (left), the corresponding measured fields at the same timestep (middle), and the prediction errors (right). The errors' magnitude remains small over the plate's surface, indicating that the predictions' accuracy does not depend on the location.

The thermal history of each node on the computational grid can be extracted from the acquired temperature fields. The thermal histories of a selection of nodes that result from 4000 steps (2000 s) of integration can be found in Fig. 7. The trajectories are uncertainty-aware, meaning that each prediction consists of a mean, which is derived from the most probable set of weights $\boldsymbol{\mu}(\boldsymbol{s})$, and from a confidence interval, which is approximated with Monte Carlo sampling. In this case, 100 samples were used for the approximation, although in practice, even less were necessary. In the plot, the measured histories are annotated with blue color, the most probable trajectory that the model predicts is indicated with a dark orange dotted line, the orange shadow indicates the region within which the temperature profile is confined with probability 95% (4σ) and the black dashed line is the absolute error calculated after aligning the signals with Dynamic time warping (DTW) (Giorgino, 2009). It is important to note firstly, that the thermal predictions of the most probable trajectory are consistently precise for every node, secondly that there is no sign of error accumulation over the extrapolation



(a) Evolution of weights θ_1, θ_4, θ_7, after 4 layers

(b) Evolution of weights θ_1, θ_4, θ_7, after 6 layers

Fig. 5 Evolution of parameters in different process stages. Blue lines illustrate the means of the distributions, the shadowed area represents the confidence interval (99%) and the stars represent the optimized weights on each state level, acting as samples. Notice how the most probable weights are not always identical to the sampled weights, and how the Gaussian Processes smooth out potential noise in the data (Color figure online)

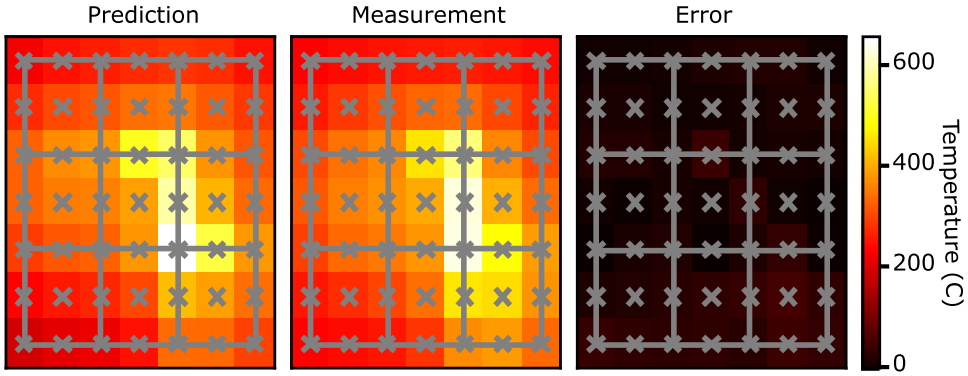


Fig. 6 Comparison between predicted and nominal thermal fields after 1650 steps (825 s) of integration. The errors between the two fields remain small over all the examined areas

horizon, and lastly that the measured temperature predictions are confined in the confidence region.

For evaluating the precision of the algorithm, after training on the thermal data stemming from one experiment, the model is validated on the remaining $n_{val} = 26$ experiments. To quantify the model's performance, we compare our predictions with measured data on the internal nodes of the plate in Fig. 3a. As a result, for every experiment, there is a total of $n_{in} = 49$ time-series to compare, yielding a total 1274 validation time-series. For every experiment, a total of $N = 4000$ integration steps ($T = 2000$ sec) is calculated. We compare the calculated results to the measured fields by using DTW to align the 2 signals and then we calculate their mean absolute relative error, $\hat{\boldsymbol{e}} = \frac{1}{N}\sum_{j=0}^{N}\left\|\frac{T_j - T_{meas_j}}{T_{meas_j}}\right\|_1$ where $\mathbf{T_{meas_j}}$ is the measured temperature field at timestep t_j, a method commonly used for comparing time-dependent data with small phase differences (Laperre et al., 2020; Ding et al., 2008). The DTW relative distance corresponds to the thermal errors the model would produce given an optimal time alignment of the predicted and measured signals. This operation allows to decouple the model performance from the delay. The errors in Fig 7 are calculated similarly using the DTW alignment between the illustrated mean prediction and measured data. The interested reader can find more details on DTW in the Appendix 8. The distribution of the means

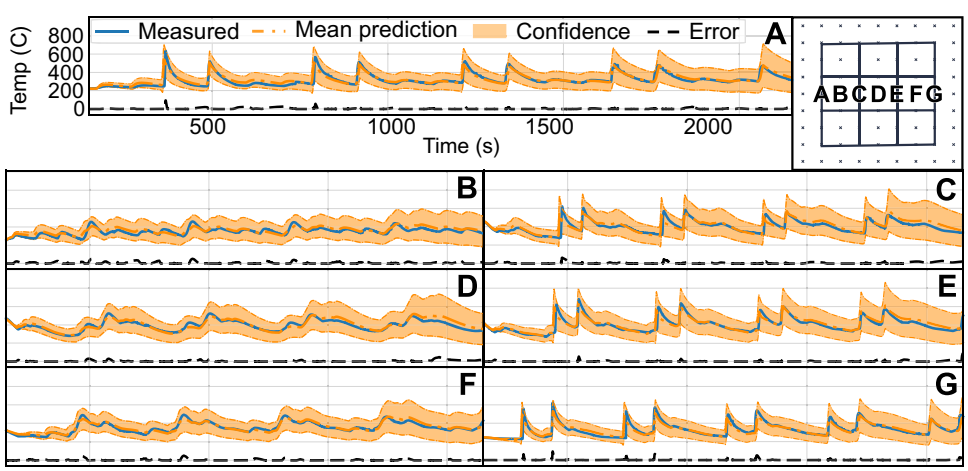


Fig. 7 Validation thermal histories of an arbitrary set of nodes. The map on the top right indicates the location of the node whose thermal history is displayed. The temperature of each node at a particular timestep is a Gaussian random variable. Note that the most probable temperature trajectories, which are defined by the mean of the Gaussians, agree with the measured ones and that the uncertainty intervals always contain the measured signal. No error accumulation is observed over the span of 2000 s of integration

of all the relative distances in every validation experiment $\hat{\boldsymbol{E}} = \{\hat{e}_{i,ex},\ i = 0 \ldots n_{in} - 1,\ ex = 0 \ldots n_{val}\}$, where $\hat{e}_{i,ex}$ is the mean relative absolute temperature error of node i in validation experiment ex, can be found in Fig. 8a. The results show a mean relative distance of *2.42%* and a variance of *1.38%*, proving the consistency of the model's performance. The distribution of the delay model errors can also be found in Fig. 8b. These errors correspond to the time offset between a predicted temperature peak and the corresponding measured one. While the vast majority of errors lie within ±20 sec, which is a small error compared to the 2000 sec of integration, few outliers are found. Outliers can be formed due to synchronization errors, problems in the interpolation of the torch path plan $\mathbf{x_t}$, or missing dynamics from the simple linear delay model.

Integrating Eq. 17 requires evaluating and adding the derivative on every single node for every timestep. This results in a linear computational complexity both in the number of nodes and time horizon. Consequently, achieving a sparse node grid and a large timestep can significantly reduce the computational burden of the integration scheme. Since the $m()$ component resembles the discretized time-domain thermal transfer function of the 2D plate, there is no need to maintain a dense grid for calculating the temperature updates as an FEM solution would require. Instead, the inter-node distances and the timestep are kept as large as possible to maintain the process's observability and save unnecessary calculations. We empirically found that we could keep a sparse grid with inter-node distances of 27 mm and use a timestep of 0.5 s. As a result, our computational scheme could execute almost *14000* times faster than the real process (0.14 s instead of 2000 s) on a *single core* of an AMD Ryzen 9 5900X, with a core bandwidth of 3,7GHz.

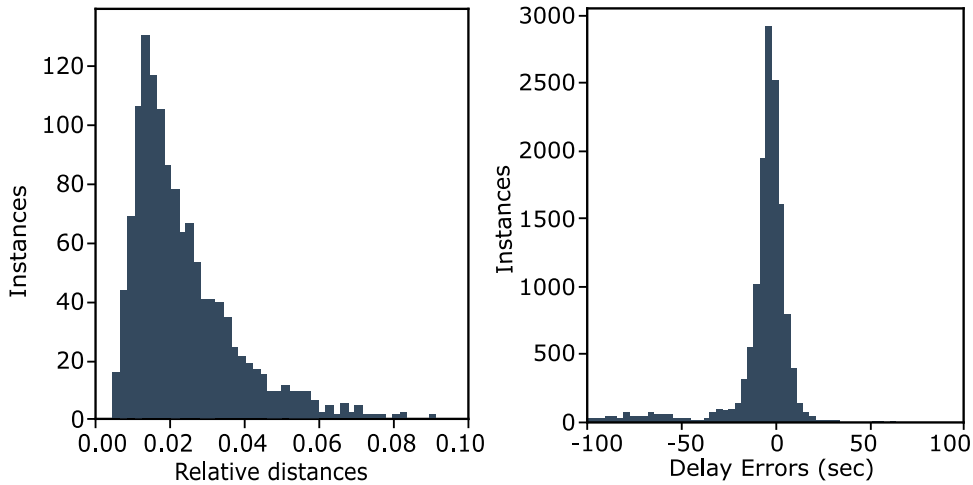
Discussion

In the present work, a new generative model family is introduced that treats the model parameters as dynamical systems, evolving in time according to a learned law. As a result, the models can adapt to multiple scenarios and learn to distinguish which system representation fits best to each case. It is straightforward to adapt the model family introduced to diverse physical domains, as illustrated in "Model for WAAM" section, where the model was adapted to solve the heat equation.

Moreover, the performance of the adapted model demonstrates that in the setting of WAAM it is feasible to learn thermal models from single experiments, even during the process execution time. Previous approaches in this setting used models that required large amounts of data. For instance, to develop a GRU Network, 250000 examples were used for training in Mozaffar et al. (2018), while in Pham et al. (2021) the authors achieve high performance after creating a dataset comprised by 20 million examples. Gathering these datasets is impractical and expensive, therefore, synthetic data were employed, obtained after lengthy simulation runs. However, the resulting models were not tested with data directly measured from an experiment and could be biased due to an extensive simulation-to-reality gap. In contrast to this, our model needs significantly less data to acquire an accurate representation of the system dynamics because of the simplicity of the architecture, the physics-informed component, and the expressive power of the Gaussian processes, which capture the evolution of the parameters in Eq. 16.

The reduction in the training data removes the need for synthetic datasets and only uses measurements stemming from a single experiment executed on the setup in Fig. 2. In this study, we developed GPyro based on the experimental results available. A comparison to other data-driven models has not been included in this work. After studying the performance of Long short-term memory (LSTM) and GRU networks on the available data set of 27 experiments, we reached the conclusion that those models are not competitive to GPyro. Due to its architecture, our model is able to generalize with data from just a single experiment, whereas purely data-driven models seem to require very large data sets to generalize. Other Physics informed neural networks (PINN) could present a similar performance as GPyro, especially soft-constraint PINNs (Raissi et al., 2019). These models incorporate physical laws and boundary conditions as terms in the loss function and in this way, they manage to fit a fully connected neural network to learn an accurate representation of the system dynamics from only a few training examples (Cai et al., 2021). However, uncertainty quantification has not been greatly explored for these models, optimization is usually performed with l-BFGS-b, which for large sets of parameters can be challenging to initialize (Lou et al., 2021), and they also present various convergence problems, even for simple dynamical systems, such as those in convection problems. In fact, Krishnapriyan et al. (2021) show that for a wide range of physical parameters, learning a PINN for such simple problems can be very challenging because of the resulting optimization landscape. Although these disadvantages demotivate the choice of a PINN for a general purpose tool like GPyro, a comprehensive comparison between the model introduced and other PINNs is beyond the scope of the current work.

One of the direct consequences of the high accuracy of the model predictions presented is the long integration intervals achieved without error accumulation. Although many accurate models have been proposed, few works have evaluated their results in long prediction horizons. The integration horizon is kept short because even the slightest prediction errors can have a substantial effect when many predictions are taken into account. However, with GPyro, we can inte-



(a) DTW relative distances **(b)** Delay model errors

Fig. 8 **A** Illustrates the Dynamically time warped (DTW) relative absolute temperature errors between the measured and the predicted temperature profiles of the validation set. The low mean and variance of the distances prove that the model is consistently performing well over all the 26 experiments of the validation set. **b** shows the delay model errors in seconds. Negative delay errors refer to prediction lags, and positive to leads. Delay errors are mostly within a range of 20 s, which is about 1% of the total extrapolation length. The outliers are caused by $\mathbf{x_t}$ interpolation errors

grate over periods of thousands of seconds, corresponding to multiple deposited layers. This is due to the linking of the thermal evolution of all nodes and the physics-informed regressor, which help maintain the balance between lost and acquired heat. As a result, the integration intervals are kept long without any substantial error accumulation. In fact, the only integration limit present when validating the models was the length of the executed experiments.

Another major advantage of GPyro is that it is ideal for process optimization and control purposes. Several model properties support this trait. One of the main strong points of GPyro is that it can be interpreted as a linear time-variant system, given its autoregressive form, the linearity of the operations between weights and features, and the smooth transitions from $m()$ to $g()$. This interpretation indicates that such a model can be used directly to shape control loops with classical control theory techniques (Middleton & Goodwin, 1988). The control-loop shaping capabilities of the model are further strengthened by the tractability in the quantification of the parameters' uncertainty that the Gaussian processes offer. The uncertainty can be leveraged for designing robust advanced control strategies such as H_∞ and robust Model predictive control (MPC) controllers. Furthermore, integrating the system evolution on sparse grids with large timesteps results in a linear decrease in computational burden and solution times that are orders of magnitude lower than the corresponding process time. Hence, the present work is well suited for closed-loop control and iterative probabilistic optimization.

The present model shows high performance in predicting the thermal fields over time horizons extending to several layers. Both validation and training data are gathered from welding the geometry illustrated in Fig. 1a with different welding paths. A valuable extension of the current framework could be an automated grid generation algorithm capable of dealing with the sparsity-retained information trade-off. Moreover, extending the current model to non-regular grids would increase the flexibility of our algorithm. This can be achieved by conditioning the parameters on the nodal distances. Furthermore, future work should be focused on improving the delay estimations, for example by designing a more robust experiment, log the robot trajectory in high resolution, and find a more sophisticated model to describe the delay dynamics. Last but not least, jointly optimizing all the parameters θ for every s would be an important addition to GPyro. This could be achieved by either using the reparametrization trick, a technique commonly used in deep Gaussian processes (Salimbeni & Deisenroth, 2017), or by forming a custom made likelihood function.

Conclusions

This paper introduces a novel generative model family, namely GPyro, and its application for predicting the temperature profiles induced by different path plans in WAAM. The following conclusions can be drawn from the previous analysis:

- GPyro is capable of learning probabilistic dynamical models from a single experiment, even in an online manner.
- Mixing simple physics-informed models with parametric surrogates whenever needed can drastically decrease the amount of data necessary for training.
- The training time of the model can be reduced to a few seconds by breaking down the optimization problem into pieces and then treating the learned parameters as samples from a joint distribution.

- Gaussian Processes can be employed to capture the time-variant nature of the heat equation. Moreover, these statistical models offer a confidence region bounding the predicted temperature evolutions.
- Modeling the interactions between nodes of a grid allows the creation of sparse grids, the use of large timesteps, and the linear decrease in the computational complexity of integration schemes, yielding them many orders of magnitude faster than reality.

Funding Open access funding provided by Swiss Federal Institute of Technology Zurich

Declarations

Competing interest The authors have no competing interests to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Appendix A: Dynamic time warping

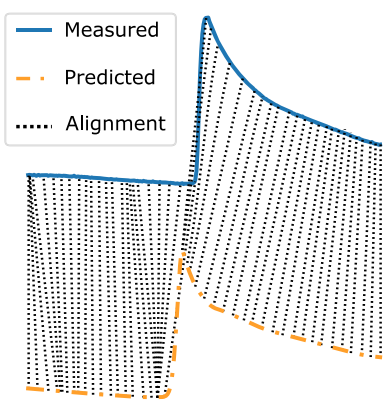


Fig. 9 DTW alignment between measured and predicted temperature profiles. The two profiles are separated with an artificial offset for reasons of clarity. DTW finds and compares similar features, decoupling the error from time shifts. This allows to model the absolute temperature error separately from the delays. Note that the dotted lines never cross each other due to the monotonicity of the alignment

Dynamic Time Warping (DTW) is a family of algorithms for comparing data with sequential dependencies, like time-series. The main goal of DTW is to find the optimal alignment between two sequences, by matching the indexes of similar features. The alignment is constrained to be monotonous, meaning that the order of indexes is maintained and that the warped index sequences never decrease. The output of DTW algorithms can be interpreted as the distance between the pairs of the most similar points.

Figure 9 illustrates the alignment between a predicted and a measured temperature signal, with an additional offset introduced between the signals, which was added for increasing the alignment visibility. In this plot, the correspondence between the points of the two signals is annotated with black dotted lines. Regardless of the time-lead present in the response, DTW manages to match the predicted and measured peak temperatures as well as the corresponding values for the cooling and the valley phases. This allows to form the relative temperature errors between similar points. In the present work, we use the absolute thermal relative errors after aligning the signals with DTW because this enables the decoupling of the thermal model's performance from the simple and weak delay model.

References

Bambach, M., Fügenschuh, A., Buhl, J., et al. (2020). Mathematical modeling and optimization for powder-based additive manufacturing. *Procedia Manufacturing, 47*, 1159–1163. https://doi.org/10.1016/j.promfg.2020.04.158

Berkenkamp, F., Schoellig, A. P.,& Krause, A. (2016). Safe controller optimization for quadrotors with gaussian processes. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 491–496. https://doi.org/10.1109/ICRA.2016.7487170

Berkenkamp, F., Turchetta, M., Schoellig, A., et al. (2017). Safe model-based reinforcement learning with stability guarantees. In Guyon, I., Luxburg, U. V., Bengio, S., et al (eds) *Proceedings of the 31st international conference on neural information processing systems*, Vol. 30. Curran Associates, Inc. https://doi.org/10.48550/arXiv.1705.08551

Bertsimas, D., Brown, D. B.,& Caramanis, C. (2010). Theory and applications of robust optimization. arXiv:1010.5445. https://doi.org/10.48550/arXiv.1010.5445

Byrd, R. H., Lu, P., Nocedal, J., et al. (1995). A limited memory algorithm for bound constrained optimization. *SIAM Journal on Scientific Computing, 16*(5), 1190–1208. https://doi.org/10.1137/0916069

Cai, S., Wang, Z., Wang, S., et al. (2021). Physics-informed neural networks for heat transfer problems. *Journal of Heat Transfer*. https://doi.org/10.1115/1.4050542

de Bruin, T., Verbert, K., & Babuska, R. (2017). Railway track circuit fault diagnosis using recurrent neural networks. *IEEE Transactions on Neural Networks and Learning Systems, 28*(3), 523–533. https://doi.org/10.1109/TNNLS.2016.2551940

Deisenroth, M. P., Fox, D., & Rasmussen, C. E. (2015). Gaussian processes for data-efficient learning in robotics and control. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 37*(2), 408–423. https://doi.org/10.1109/TPAMI.2013.218

Deringer, V. L., Bartók, A. P., Bernstein, N., et al. (2021). Gaussian process regression for materials and molecules. *Chemical Reviews, 121*(16), 10073–10141. https://doi.org/10.1021/acs.chemrev.1c00022

Ding, H., Trajcevski, G., Scheuermann, P., et al. (2008). Querying and mining of time series data: Experimental comparison of representations and distance measures. *Proceedings of the VLDB Endowment, 1*(2), 1542–1552. https://doi.org/10.14778/1454159.1454226

Gh Ghanbari, P., Mazza, E., & Hosseini, E. (2020). Adaptive local-global multiscale approach for thermal simulation of the selective laser melting process. *Additive Manufacturing, 36*(101), 518. https://doi.org/10.1016/j.addma.2020.101518

Giorgino, T. (2009). Computing and visualizing dynamic time warping alignments in r: The dtw package. *Journal of Statistical Software, 31*(7), 1–24.

Goldak, J., Chakravarti, A., & Bibby, M. (1984). A new finite element model for welding heat sources. *Metallurgical Transactions B, 15*(2), 299–305. https://doi.org/10.1007/BF02667333

Graf, M., Hälsig, A., Höfer, K., et al. (2018). Thermo-mechanical modelling of wire-arc additive manufacturing (waam) of semi-finished products. *Metals*. https://doi.org/10.3390/met8121009

Gu, J., Gao, M., Yang, S., et al. (2020). Microstructure, defects, and mechanical properties of wire + arc additively manufactured alcu4.3-mg1.5 alloy. *Materials & Design* 186:108357. https://doi.org/10.1016/j.matdes.2019.108357

Hejripour, F., Binesh, F., Hebel, M., et al. (2019). Thermal modeling and characterization of wire arc additive manufactured duplex stainless steel. *Journal of Materials Processing Technology, 272*, 58–71. https://doi.org/10.1016/j.jmatprotec.2019.05.003

Hewing, L., Kabzan, J., & Zeilinger, M. N. (2020). Cautious model predictive control using gaussian process regression. *IEEE Transactions on Control Systems Technology, 28*(6), 2736–2743. https://doi.org/10.1109/TCST.2019.2949757

Jain, A., Nghiem, T., Morari, M., et al. (2018). Learning and control using gaussian processes. In: *2018 ACM/IEEE 9th international conference on cyber-physical systems (ICCPS)*, pp. 140–149, https://doi.org/10.1109/ICCPS.2018.00022

Krishnapriyan, A. S., Gholami, A., Zhe, S., et al. (2021). Characterizing possible failure modes in physics-informed neural networks. arXiv:2109.01050. https://doi.org/10.48550/arXiv.2109.01050

Laperre, B., Amaya, J., & Lapenta, G. (2020). Dynamic time warping as a new evaluation for dst forecast with machine learning. *Frontiers in Astronomy and Space Sciences,*. https://doi.org/10.3389/fspas.2020.00039

Le, V. T., Nguyen, H. D., Bui, M. C., et al. (2022). Rapid and accurate prediction of temperature evolution in wire plus arc additive manufacturing using feedforward neural network. *Manufacturing Letters, 32*, 28–31. https://doi.org/10.1016/j.mfglet.2022.02.003

Lee, S. H. (2020). Optimization of cold metal transfer-based wire arc additive manufacturing processes using gaussian process regression. *Metals*. https://doi.org/10.3390/met10040461

Lou, Q., Meng, X., & Karniadakis, G. E. (2021). Physics-informed neural networks for solving forward and inverse flow problems via the Boltzmann-Bgk formulation. *Journal of Computational Physics, 447*(110), 676. https://doi.org/10.1016/j.jcp.2021.110676

Middleton, R., & Goodwin, G. (1988). Adaptive control of time-varying linear systems. *IEEE Transactions on Automatic Control, 33*(2), 150–155. https://doi.org/10.1109/9.382

Montevecchi, F., Venturini, G., Scippa, A., et al. (2016). Finite element modelling of wire-arc-additive-manufacturing process. *Procedia CIRP, 55*, 109–114. https://doi.org/10.1016/j.procir.2016.08.024

Mozaffar, M., Paul, A., Al-Bahrani, R., et al. (2018). Data-driven prediction of the high-dimensional thermal history in directed energy deposition processes via recurrent neural networks. *Manufacturing Letters, 18*, 35–39. https://doi.org/10.1016/j.mfglet.2018.10.002

Mozaffar, M., Liao, S., Lin, H., et al. (2021). Geometry-agnostic data-driven thermal modeling of additive manufacturing processes using graph neural networks. *Additive Manufacturing, 48*(102), 449. https://doi.org/10.1016/j.addma.2021.102449

Neiva, E., Badia, S., Martín, A. F., et al. (2019). A scalable parallel finite element framework for growing geometries. Application to metal additive manufacturing. *International Journal for Numerical Methods in Engineering, 119*(11), 1098–1125. https://doi.org/10.1002/nme.6085

Ness, K. L., Paul, A., Sun, L., et al. (2022). Towards a generic physics-based machine learning model for geometry invariant thermal history prediction in additive manufacturing. *Journal of Materials Processing Technology, 302*(117), 472. https://doi.org/10.1016/j.jmatprotec.2021.117472

Nguyen, L., Buhl, J., Israr, R., et al. (2021). Analysis and compensation of shrinkage and distortion in wire-arc additive manufacturing of thin-walled curved hollow sections. *Additive Manufacturing, 47*(102), 365. https://doi.org/10.1016/j.addma.2021.102365

Pascanu, R., Mikolov, T.,& Bengio, Y. (2012). On the difficulty of training recurrent neural networks. arXiv:1211.5063. https://doi.org/10.48550/arXiv.1211.5063

Pham, T., Hoang, T., Pham, Q., et al. (2021). Data-driven prediction of temperature evolution in metallic additive manufacturing process. In *International conference on material forming (ESAFORM)*.

Priarone, P. C., Pagone, E., Martina, F., et al. (2020). Multi-criteria environmental and economic impact assessment of wire arc additive manufacturing. *CIRP Annals, 69*(1), 37–40. https://doi.org/10.1016/j.cirp.2020.04.010

Raissi, M., Perdikaris, P., & Karniadakis, G. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics, 378*, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

Ranzato, M., Chopra, S., Auli, M., et al. (2015). Sequence level training with recurrent neural networks. arXiv:1511.06732

Rasmussen, C. E., & Williams, C. K. I. (2005). Gaussian processes for machine learning. The MIT Press. https://doi.org/10.7551/mitpress/3206.001.0001

Ren, K., Chew, Y., Zhang, Y., et al. (2020). Thermal field prediction for laser scanning paths in laser aided additive manufacturing by physics-based machine learning. *Computer Methods in Applied Mechanics and Engineering, 362*(112), 734. https://doi.org/10.1016/j.cma.2019.112734

Roberts, S., Osborne, M., Ebden, M., et al. (2013). Gaussian processes for time-series modelling. *Philosophical Transactions of the Royal Society A* 371(1984):20110550. https://doi.org/10.1098/rsta.2011.0550

Roy, M., & Wodo, O. (2020). Data-driven modeling of thermal history in additive manufacturing. *Additive Manufacturing, 32*(101), 017. https://doi.org/10.1016/j.addma.2019.101017

Sahli Costabal, F., Matsuno, K., Yao, J., et al. (2019). Machine learning in drug development: Characterizing the effect of 30 drugs on the qt interval using gaussian process regression, sensitivity analysis, and uncertainty quantification. *Computer Methods in Applied Mechanics and Engineering, 348*, 313–333. https://doi.org/10.1016/j.cma.2019.01.033

Salimbeni, H.,& Deisenroth, M. (2017). Doubly stochastic variational inference for deep gaussian processes. In Guyon, I., Luxburg, U. V., Bengio, S., et al. (Eds.), *Advances in neural information processing systems*, Vol. 30. Curran Associates, Inc., https://doi.org/10.48550/ARXIV.1705.08933

Schubnel, B., Carrillo, R. E., Taddeo, P., et al. (2020). State-space models for building control: How deep should you go? *Journal*

of Building Performance Simulation, 13(6), 707–719. https://doi.org/10.1080/19401493.2020.1817149

Stathatos, E.,& Vosniakos, G.C. (2019). Real-time simulation for long paths in laser-based additive manufacturing: A machine learning approach. *The International Journal of Advanced Manufacturing Technology* 104:1967–1984. https://doi.org/10.1007/s00170-019-04004-6

Stockman, T., Schneider, J. A., Walker, B., et al. (2019). A 3D finite difference thermal model tailored for additive manufacturing. *JOM, 71*(3), 1117–1126. https://doi.org/10.1007/s11837-019-03338-6

Taylor, J. S. (2001). *An engineer's guide to fabricating steel structures / by John Taylor*. Australian Institute of Steel Construction North Sydney

Xiong, J., Li, R., Lei, Y., et al. (2018). Heat propagation of circular thin-walled parts fabricated in additive manufacturing using gas metal arc welding. *Journal of Materials Processing Technology, 251*, 12–19. https://doi.org/10.1016/j.jmatprotec.2017.08.007

Zhou, Z., Shen, H., Liu, B., et al. (2021). Thermal field prediction for welding paths in multi-layer gas metal arc welding-based additive manufacturing: A machine learning approach. *Journal of Manufacturing Processes, 64*, 960–971. https://doi.org/10.1016/j.jmapro.2021.02.033

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.